Université Sorbonne Paris Nord

L1 Informatique
Année 2025-2026

Mini-projet du module
Programmation Impérative 2 (Info/DL)

Jeux de tir

Consignes

Mini-Projet à réaliser en binôme (ou en monôme) au sein de votre groupe de TP.

Vous devez avoir indiqué votre binôme avant la fin de la première séance de la semaine du 13 avril à votre chargé.e de TP.
En cas d'oublis ou d'absence, confirmez cela par mail avec le.a chargé.e de TP !

Vous devez rendre par l'ENT, avant le dimanche 17 mai 2026 23h59 une archive contenant

- votre code (fichier .h, .c et Makefile uniquement) et éventuelles fichiers de données
- un fichier README.txt indiquant le nom des auteurs (votre monôme/binôme), description du programme, usage du programme, caractériques de votre code... et tout ce que vous jugez utile à transmettre avec rendu (difficultés rencontrées, bugs corrigés ou non etc).

Lors des séances de TP de la semaine du 18 mai 2026, vous devez venir présenter, lors d'une mini-soutenance,

- une exécution de votre code de la partie A du Mini-Projet
- répondre à des questions concernant des lignes de code précises et l'organisation globale du code de la partie A.
- présenter votre partie B, exécution et réponses à des questions.

Sur les 3 séances de 3h de TP, votre chargé de TP prendra des notes sur votre progression et pourra vous aider et vous guider.

Cadre du mini-projet

Le jeu se déroule dans une grille 2D rectangulaire de $n \times m$ cases, entouré d'un cadre. On s'intéresse aux jeux de tirs, appelés en anglais *Shoot'em up*[1] où un canon se déplace depuis le bas de l'écran lorsque le joueur appuie sur les flèches du clavier. On distingue deux types de jeux de tirs

- les "Shoot'em up fixe" où le canon ne se déplace qu'horizontalement lorsque joueur appuie sur une flèche : ce sera le cadre de la partie A avec le tir aux canards, et surtout du jeu historique Space Invaders
- les "Shoot'em up mobile" où le canon est plutôt vu comme un avion vu d'au-dessus qui peut se déplacer un peu partout sur l'écran.

Dans les deux cas, des objets à éliminer apparaissent dans la partie haute de l'écran et le canon doit tirer dessus avec des projectiles partant du canon afin de les éliminer du jeu.

Ce projet se divise en deux parties :

- Partie A : cette partie est un TP classique avec des questions dans le but de construire un jeu dits de '"Tirs aux canards", shoot'em up fixe simple.
- Partie B : sur la base de la partie A (à finir au plus tôt donc !), vous développerez un jeu de déplacement plus élaboré.

Le rendu et la soutenance portent sur les deux parties !

1. Ce jeu historique est une illustration de plus de la nature belliqueuse des jeux vidéos, ce qui est regrettable à une époque où l'humanité redécouvre les conflits militaires généralisés et où on prend en considération le bien être animal. Vous pouvez tout à fait remplacer le canon par un pistolet à eau, un envoi de cœurs... et les cibles par des fantômes ou des animaux vers lesquels sont projetées des vitamines...

Exercice d'introduction

Exercice 0 : Prise en main d'un exemple minimal

Les réponses à cet exercice 0 n'ont pas à être rédigée à l'écrit, son but est de vous guider dans le projet et la découverte du code d'exemple : attention des questions sur les notions de cet exercice peuvent être à l'origine d'une question lors à l'oral).

Décompresser l'archive `Jeu_exemple.tgz` qui contient
- `README.txt` : qui donne les indications sur le jeu
- `Makefile`
- `main_jeu.c` : qui contient le main et la boucle événementielle
- `Canon.h` et `Canon.c` qui contiennent les fonctions associés à un canon.

Liser le README qui vous indique comment installer et joueur. Installer le jeu et jouer. Vous pouvez constater que ce jeu est "minimal" (est-ce un jeu ?...) : en appuyant sur les flèches du clavier, le joueur fait bouger à gauche et à droite le canon.

Q 0.1**.** Déplacer le canon avec les flèches et appuyez sur ESC pour sortir. Essayer le jeu avec différentes taille d'écran (réduiser éventuellement la police du terminal si besoin).

Le but de ce programme est de vous fournir les outils nécessaire à un jeu de tirs.

Il y a cinq outils utilisés ici
- le paradigme de la programmation événementiel qui est la base de la plupart des jeux vidéos où des éléments se déplacent
- la libraire `ncurses` pour utilser les flèches du clavier
- le principe d'un ensemble coordonné de struct et de fonctions, [2], rassemblé en deux fichiers .h et .c, permettant de créer, piloter, afficher et intéragir avec un élément du jeu : ici un canon.
- l'affichage Texte
- Le graphisme Texte avec l'ASCII ART, les séquences d'échappements ANSI et les émojis Unicode

Programmation événementielle

Le principe de la programmation événementielle repose sur le fait de scruter très fréquemment les évènements extérieurs au programme et d'activer des lignes de codes en réaction avec ces évènements.
Ici les vénements sont principalement le fait que le joueur appuie sur une touche que l'on récupère par la commande getch.

Pour mettre en œuvre la programmation évènementielle, on utilise une boucle qui est le

2. appelé objet en programmation objet

cœur du programme. Cette boucle commence par récupérer la saisie éventuelle d'une touche par l'utilisateur. Si aucune touche n'a été appuyée, le programme se poursuit en considérant qu'aucune action n'a été faite : par exemple, le canon reste immobile alors que des éléments du jeu bougent. Si une touche a été pressée, les conséquences de cette pression sont évaluées, entraînant un changement d'affichage : par exemple permettant de déplacer le canon.

Q 0.2. Repérer la boucle dans le programme exemple et les endroit de la boucle correspondant aux réactions en cas d'appui sur les toûches. Expliquer comment la touche ESC permet de sortir du jeu. Si vous vouliez ajouter une touche secrète (cheat code), comme l'appui de la lettre V par exemple, qui permettrait de gagner le jeu, comment modifier le programme ?

Librairie ncurses

Pour utiliser les flèches du clavier, le programme utilise la librairie `ncurses.h` [3]

Attention pour utiliser cette librairie (comme pour math.h), il faut effectuer la dernière étape de compilation avec l'option `-lncurses`.

Nous utilisons ici uniquement les aspects de gestion des interruptions système correspondant au clavier de ncurses (qui a de nombreuses autres fonctionnalités). Une interruption système désigne le fait qu'un programme est mis en pause par le micro-processeur quelques instants pour effectuer une autre tâche : ici cette autre tâche est de récupérer une éventuelle frappe sur le clavier.

Contrairement à `scanf`, nous allons utiliser `getch` qui renvoie un code dès qu'une touche a été utilisée sur le clavier. Si vous utilisez `getch` sans autre commande préalable, le programme est mis en attente jusqu'à ce qu'une touche soit appuyée. Mais dans l'exemple fourni, la commande `halfdelay(x)` force `getch` a s'interrompre au bout de `x` dixièmes de seconde [4]. Le programme retrouve alors la main pour faire autre chose : ici dans l'exemple, on traite alors dans la boucle quelle touche a été utilisée. On peut faire varier ce temps `x` pour avoir un jeu plus ou moins rapide pour le joueur. [5].

Attention, une étrangeté de ncurses fait que, parfois, après la première utilisation de getch, l'écran devient noir... Il faut donc plutot afficher après l'utilisation de getch.

Le reste des commandes de `ncurses` utilisées dans l'exemple sont nécessaires pour faire fonctionner `getch` en lien avec la fenêtre du terminal. Pensez à toujours les mettre en début et fin de programme (si vous les oublier en fin de programme, le terminal va dysfonctionner par la suite).

3. Cette librairie est installé sur les machines de Galilée. Vous pouvez l'installer sous linux en utilisant les commandes `sudo apt-get install libncurses5-dev libncursesw5-dev`. Si vous voulez utiliser des curseurs sous windows, un équivalent à `ncurses` existe avec `conio.h` que vous pouvez trouver ici par exemple `https://www.develop4fun.fr/la-librairie-conio-h-_kbhit-et-getch/`

4. Sous windows avec la librairie `conio.h`, la commande proche de `halfdelay` est `nodelay(win, TRUE);`

5. Remarquer que cette utilisation empêche de descendre en dessous du dixième de seconde pour la réactivité du programme et la valeur 1 est souvent la bonne. Pour un code plus réactif, il faudait directement gérer les interruptions système avec des commandes système comme le propose par exemple les librairies SDL ou syst que l'on a sous Linux

Q 0.3. Quelle valeur retournée par getch correspond à l'absence de pression sur une touche pendant une itération du jeu ?

L'ensemble de structs et de fonctions : Canon.h et Canon.c

Chaque élément d'un jeu peut être encodé par un un struct pour contenir ses données et des fonctions pour gérer son fonctionnement. Dans l'exemple, un Canon est codé comme un struct permettant d'avoir

- des informations fixes comme celles sur son affichage ou ses positions limites à l'écran
- des informations qui changent au cours du jeu comme sa position à l'instant ou sa position juste avant son déplacement.

et des fonctions permettant

- intialisation/désollocation
- les conséquences de l'appui d'une touche sur sa position
- et surtout son affichage à l'écran

Q 0.4. A quoi sert le champs `pas` dans le struct `Canon`

Affichage Texte

L'affichage à l'écran est ici en mode Texte dans le terminal. Ainsi, l'écran est un ensemble de cases rectangulaires, toutes de même taille, pouvant afficher des caractères textes.
L'ensemble de struct et de fonctions codées dans `Affiche.h` et `Affiche.c` contient ainsi

- les informations sur la taille de l'écran de jeu : c'est-à-dire un nombre de cases situés dans le terminal : si ce nombre est trop grand, il faut réduire la police du terminal (CTRL-)
- un tableau `tab` à deux dimensions tel que `tab[i][j}` indique ce qui doit être affiché dans la case `(i,j)`. On va voir dans la suite qu'il faut parfois plusieurs caractères pour indiquer un affichage d'une unique case : ainsi `tab[i][j]` est une chaîne de caractères.

et des fonctions permettant

- de dessiner un cadre tout autour de l'écran en plaçant des cases de couleurs sur les bords du tableau `tab`
- de vider le tableau en mettant un espace dans chaque case
- d'afficher le tableau à l'écran avec la commande `printf` pour chaque case : la chaînes de caractères `tab[i][j]` devant contenir une séquence correspondant à une unique case !

Q 0.5. Pourquoi la fonction vider consiste en fait à remplir le tableau d'espaces à chaque case et non d'une chaîne vide `""` ?

Le graphisme Texte avec l'ASCII ART, les séquences d'échappements ANSI et les émojis Unicode

Si l'on reste dans la logique d'une case par éléments, on ne fait bouger que des éléments rectangulaires ou des lettres, chiffres... Pour pouvoir réaliser des dessins agréables à regarder en mode texte, on peut avoir recours aux trois outils graphiques suivants :

- **l'ASCII ART :** qui consiste à n'utiliser que des caractères ASCII (donc monochrome), mais de manière assez jolie pour permettre un affichage surprenant expressif et beau. Ce système est assez économique car il ne demande qu'un caractère par case. Par contre, il faut plusieurs cases pour un dessin un peu visible. L'exemple ultra-célèbre des Shoot'em up sont les aliens de Space Invader ! Pour utiliser cette idée, on peut stocker un dessin representé comme une matrice de caractères ASCII que l'on peut stocker dans un fichier. Cette matrice est alors stockée dans un struct dédié à l'éléments et sera affichér autant de fois que nécessaire.
  Référence utile : `https://fr.wikipedia.org/wiki/Art_ASCII`
- **ANSI Escape Sequences** [6] **:** les "séquences d'échappements ANSI" sont des suites de caractères ASCII commençant par le caractère ASCII 27 correspondant à ESC, qui peut être écrit (en octal) par \33 : on fait suivre immédiatement ce code d'autres caractères qui composent une commande. Ces caractères ne sont pas affichés mais ils ont une action sur le terminal : "\33[2J" : efface le terminal ; "\33[H" : déclare que le prochain caractère sera écrit en haut à gauche du terminal ;
- "\33[XXm" où XX est un nombre décimal : déclare que les caractères suivants sont de couleur de fonds noir si XX=00 (rouge si XX=41, vert si XX=42...).
  A noter que "\33[1E" qui va en début de la ligne suivante **A préférer à** $\backslash n$.
  Référence utile : `https://gist.github.com/fnky/458719343aabd01cfb17a3a4f7296797`.
- **Emojis Unicode** La norme Unicode permet d'étendre considérablement les caractères textes, proposant des signes et des émojis divers. Attention, ces codes ne sont pas universels au sens où tous les outils ne les reconnaissent pas : ils sont alors affichés comme une suite de plusieurs caractères disparates (ou alors comme un caractère unique carré portant de 0 et de 1). Et entre deux outils qui les reconnaissent, ils peuvent de pas avoir un affichage identique.
  Référence utile :

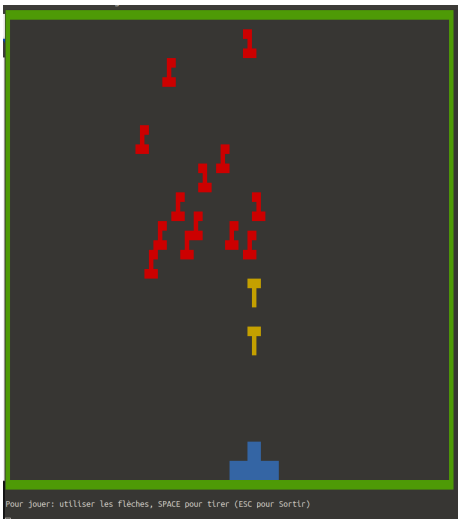
Attention ! ! ! Il est important de noter que les séquences d'échappement sont codées sur plusieurs octets : il faut une séquence de 5 `char` pour coder une seule case de séquence ! Bien plus encore, les émojis Unicode peuvent consommer encore plus. Dans le code exemple, une case du tableau `tab` est définie comme une séquence de 5 caractères, il faut peut-être une allocation plus importantes pour certains émojis. Heureusement, les terminaux linux ont de plus en plus un affichage unicode très correct.

Q 0.6**.** Modifier l'affichage du canon pour le couvrir d'un ou plusieurs emojis coeur (ou autre choix poétique).

6. Il est à noter que ces séquences datent de l'époque ds téléscripteurs pour formater les pages papier issus des premiers terminaux.

Partie A : Tir aux canards

Voici une capture d'écran d'un jeu de tir aux canards



Ici les canards sont des représentations (dessinées par du ASCII art coloré avec talent) de cibles (souvent en bois) en forme de canard passant de gauche à droite, ou de droite à gauche. Le canon envoie des flèches sur les cibles pour les faire tomber pour qu'elles disparaissent toutes. Vous pouvez être tenté d'améliorer ce jeu dès à présent... Ce n'est pas interdit ! En revanche, il est tout à fait autorisé de changer l'aspect esthétique (et de faire un canon à cœur envoyés sur des têtes d'oursons en peluches).

Mais **attention**, il faut que la Partie A marche et que vous repectiez l'énoncé ci-dessous (avec des listes chaînées etc).

Exercice 1 : Fléchettes

On veut coder une structure et des fonctions permettant de gérer les fléchettes

Q 1.1. Créer un fichier `Flechette.h` et `Flechette.c` définissant :
une structure nommée `Liste_Flechette` contenant

- les informations sur la taille des fléchettes et leur représentations
- la coordonnée verticale d'où partent les fléchettes et la coordonnée maximale en haut où elles disparaissent
- une liste simplement chaînée de `Flechette` où une `Flechette` est un maillon de la liste. Chacun des éléments est une des fléchettes et indique les coordonnées du point en haut à gauche de la fléchette.

On peut ajouter d'autres informations dans `Liste_Flechette` : par exemple un nombre max de fléchettes et un compteur du nombre de fléchettes si on veut limiter cette quantité au cours du jeu.

Q 1.2. Ecrire les fonctions

```
Liste_Flechette* Liste_Flechette_initialiser_vide(int hauteur_aff, int nivdiff);
void Flechette_desallouer(Liste_Flechette** Lflech);
```
pour allouer une liste de 0 fléchettes et désallouer une liste (contenant éventuellement plusieurs fléchettes). Le niveau de difficultés peut se traduire en un nombre maximal de fléchettes disponibles en même temps par exemple.

Q 1.3. Ecrire la fonction
```
void ajouter_flechette(Liste_Flechette* Lflech, int y);
```
qui ajoute une fléchette en position horizontale y (a priori en sortie du canon) : cette fléchette est insérée en tête de la liste chaînée.

Q 1.4. Modifier le main pour
- définir au début une liste de fléchettes vides
- créer une fléchette à l'emplacement du canon si le joueur appuie sur espace (On peut remarquer que ce n'est pas vraiment le canon qui envoie la fléchette d'un point de vue activité du `Canon`, mais bien le `main` qui ajoute une fléchette à l'emplacement du canon).

Q 1.5. Ecrire la fonction
```
void Flechette_afficher(Liste_Flechette *Lflech, Affichage* A);
```
qui parcourt la liste des fléchettes pour les placer dans le tableau d'affichage `tab`.
Et ajouter un appel à cet affichage dans le main.

Q 1.6. Ecrire la fonction
```
void Flechette_action(Liste_Flechette *Lflech);
```
qui fait avancer toutes les fléchettes d'une case vers le haut et qui supprime celles qui atteignent le haut de l'écran.
Et ajouter cette action à chaque itération de la boucle événementielle.

Exercice 2 : Canards

On veut coder une structure et des fonctions permettant de gérer les canards

Q 2.1. Créer un fichier `Canard.h` et `Canard.c` définissant :
une structure nommée `Liste_Canard` contenant
- les informations sur la taille des canards et leur représentations
- la coordonnée horizontales gauche et droite d'où partent et où disparaissent les canards, ainsi que le pas de déplacement
- une liste simplement chaînée de `Canard` où une `Canard` est un maillon de la liste. Chacun des éléments est un des canards et indique les coordonnées horizontal du point en haut à gauche du canard et sa direction vers la droite ou vers la gauche.
- le nombre de canard dans la liste.

Q 2.2. Ecrire les fonctions
```
Liste_Canard* Liste_Canard_initialiser_vide(int largeur_aff, int hauteur_aff,
                                            int nivdiff, char * nomfic);
void Canard_desallouer(Liste_Canard** Lcanard);
```

pour allouer une liste de 0 canardset désallouer une liste (contenant éventuellement plusieurs canards).
Le niveau de difficultés peut se traduire en un nombre max de canards. Le nom du fichier passé en paramètre peut-être la source du dessin en ASCII art du canard par exemple. Mais il peut aussi être remplacé par une chaîne de caractères correspondant à une émoji ou autre idée graphique.

Q 2.3. Ecrire la fonction
`void ajouter_canard(Liste_Canard* Lcanard);`
qui ajoute un canard si le nombre max n'est pas atteint. Pour en créer, il faut tirer au hasard s'il part vers la gauche ou la droite, ainsi que la ligne où il va avancer. Ce canard est inséré en tête de la liste chaînée.

Q 2.4. Modifier le main pour
- définir au début une liste de canards vides
- appeler la création d'un canard à chaque itération de la boucle

Q 2.5. Ecrire la fonction
`void Canard_afficher(Liste_Canard *Lcanard, Affichage* A);`
qui parcourt la liste des canards pour les placer dans le tableau d'affichage `tab`.
Et ajouter un appel à cet affichage dans le main.

Q 2.6. Ecrire la fonction
`void Canard_action(Liste_Canard *Lcanard);`
qui fait avancer tous les canards d'une case dans la direction stocké dans ce canard et qui supprime ceux qui atteignent un bord de l'écran.
Et ajouter cette action à chaque itération de la boucle événementielle.

Exercice 3 : Intéractions et jeu complet

Q 3.1. On veut ajouter la fonction
`void Interaction_Flechette_Canard(Liste_Flechette *Lflech, Liste_Canard *Lcanard);`
Où la placer ? Ecrivez-la et utilisez-la dans le main pour permettre à une fléchette de faire tomber une cible canard.
Le canard doit être enlevé de sa liste (et éventuellement aussi la fléchette aussi).

Q 3.2. Ajouter le test de victoire dans le main qui permet de sortir de la boucle.

Q 3.3. Il faut toujours prévoir une phase de tests pour un programme :
Tester votre programme dans différentes situations.

Partie B : A vous de jouer !

Dans cette deuxième partie, il vous est demandé de créer un jeu plus élaboré à partir de votre code de la partie A. **Pensez déjà à bien faire fonctionner la partie A (et à en conserver une version intacte et fonctionnelle, prête à être montrée pour l'évaluation).**

Vous pouvez choisir une variante plus ou moins compliquée !
Si elle fonctionne, vous serez évalué sur votre réalisation finale.
Si elle n'est pas achevée ou fonctionne imparfaitement, vous serez évalué sur votre ambition et ce que vous aurez mis en œuvre pour essayer d'y parvenir : n'hésitez donc pas à choisir une variante qui vous semble intéressante.

Il y a deux grands cadres possibles :

- Améliorer **fortement** la partie A : les canards deviennent des ballons qui rebondissents, il y a des obstacles, les fléchettes sont des billes qui rebondissent, il y a des canards plus rapides que d'autres, des fléchettes avec des pouvoirs, un joueurs peut perdre des fléchettes...
- Coder un autre jeu proche mais avec un changement important : Space Invaders (où les aliens sont lents mais vous bombardent en retour...) ; Shoot-'em up, le canon est un avion vu du ciel qui peut avancer vers le haut ou le bas en plus, récupérer des objets ; billard...

Il faut faire valider votre idée de partie B par votre chargé.e de TP :
- iel doit pouvoir suivre votre progression pour vous évaluer (si vous présentez quelque chose d'inconnu le dernier jour... on sera soupçonneux)
- et surtout votre chargé de TP peuvt vous conseiller, vous guider et vous aider : par exemple à ne pas prendre un projet trop ambitieux pour la partie B.